



INSTITUTO POLITECNICO NACIONAL



UNIDAD PROFESIONAL INTERDISCIPLINARIA DE INGENIERIA Y CIENCIAS SOCIALES Y ADMINISTRATIVAS

Ingeniería en Informática



ALUMN@S:

Castañeda Ibáñez Hugo Erik

Gutiérrez Gutiérrez Leslie

Mendieta Rojas Delfino

Silva Maya Angelli Monserrat

SECUENCIA: 4NM50

Profesora: Méndez García Sara

ÍNDICE

1. Introducción	
1.1.1. Lenguaje y Gramática	2
1.1.2. Compilador	3
1.1.2.1.1. Fases de un compilador completo	3
1.1.2.1.1.1. Análisis Léxico	
1.1.2.1.1.2. Análisis sintáctico	
1.1.2.1.1.3. Análisis semántico	5
1.1.3. Expresiones regulares	6
1.1.4. Puerto paralelo	
2. Caso Práctico	8
3. Objetivo General	8
4. Objetivos Específicos	9
5. Justificación	9
6. Alcances y limitaciones	10
7. Descripción general del proyecto	11
8. ¿Cómo funcionará?	12
9. ¿Cómo será el carrito?	13
10. ¿Cómo se llevará a cabo?	15
11. ¿Qué herramientas se utilizarán?	17
12. Interacción entre las herramientas	18
13. ¿Cómo será la entrega al final?	19
14. Diseño del AFND para las instrucciones	20
15. BNF de los comandos aceptados	21
16. Manual de usuario	22
17. Referencias bibliográficas	23

INTRODUCCIÓN

El presente proyecto lo iniciaremos mencionando las definiciones de:

Lenguaje:

“Es un conjunto de palabras o cadenas de determinados símbolos del propio lenguaje que permiten expresar ideas para ser comprendidos por los demás, un lenguaje formal es diseñado para expresar procesos que pueden ser llevados a cabo por máquinas como las computadoras.” (Book, 1977)¹



Gramática:

Es un conjunto de reglas que gobiernan a un lenguaje, las cuales permiten organizar de forma coherente las cadenas del mismo, para establecer un orden general de cómo expresarse en cierto contexto de forma correcta o aceptable.

El paradigma formal más famoso y rápidamente desarrollado para la caracterización de lenguajes, es el derivado del concepto de gramática generativa de Noam Chomsky.

Según Chomsky la gramática generativa de una lengua es un sistema de reglas que se agrupan en tres componentes, uno generativo y dos interpretativos.²

La Jerarquía de Chomsky consta de cuatro niveles:

¹ Book, D. (1977). Dragon Book. En A. V. Aho, *Dragon Book* (pág. 803). Dax.

² Alfred Aho, R. S. (s.f.). Compiladores . En R. S. Alfred Aho, *Compiladores* . Addison – Wesley.



JERARQUÍA DE CHOMSKY:

Gramática	Lenguaje	Modelo Matemático
Tipo 0: Irrestricta	Recursivamente enumerable (Nivel Pragmático)	Máquina de Turing (MT)
Tipo 1: Dependiente del Contexto	Dependiente del Contexto (Nivel Semántico)	Autómata Linealmente Limitado (ALL)
Tipo 2: Independiente del Contexto	Independiente del Contexto (Nivel Sintáctico)	Autómata de Pila (AP)
Tipo 3: Regular	Regular (Nivel Léxico)	Autómata Finito (AF)

INC - JORGE RUARDO

De tal modo que el conjunto de lenguajes recursivamente enumerables contiene al conjunto de lenguajes dependientes del contexto, éste contiene a los lenguajes independientes del contexto y éstos a los regulares:

$$\mathcal{L}_0 \supset \mathcal{L}_1 \supset \mathcal{L}_2 \supset \mathcal{L}_3$$

$$\mathcal{L}_0 \supset \mathcal{L}_1 \supset \mathcal{L}_2 \supset \mathcal{L}_3$$

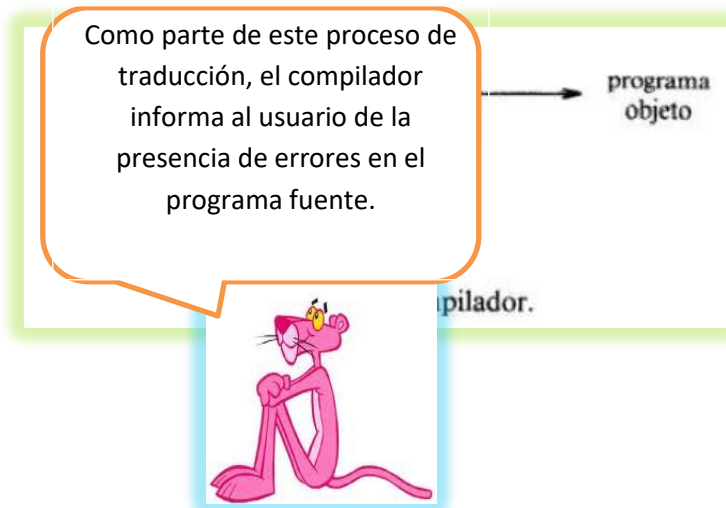
³ (GRAMÁTICA GENERATIVA, 2010)

³ GRAMÁTICA GENERATIVA. (27 de Noviembre de 2010). Recuperado el 11 de septiembre de 2013, de GRAMÁTICA GENERATIVA: <http://apunteslinguistica.blogspot.mx/2010/12/chomski-gramatica-generativa-puntos-de.html>

En este caso nosotros utilizaremos la clasificación 3 que es sobre los lenguajes regulares ya que este se resuelve mediante autómatas finitos.

Por consiguiente, se explicara que es un COMPILADOR COMPLETO.

“Es un programa que lee un programa escrito en un lenguaje, el lenguaje fuente, y lo traduce a un programa equivalente en otro lenguaje, el lenguaje objeto.” (Book, 1977)⁴

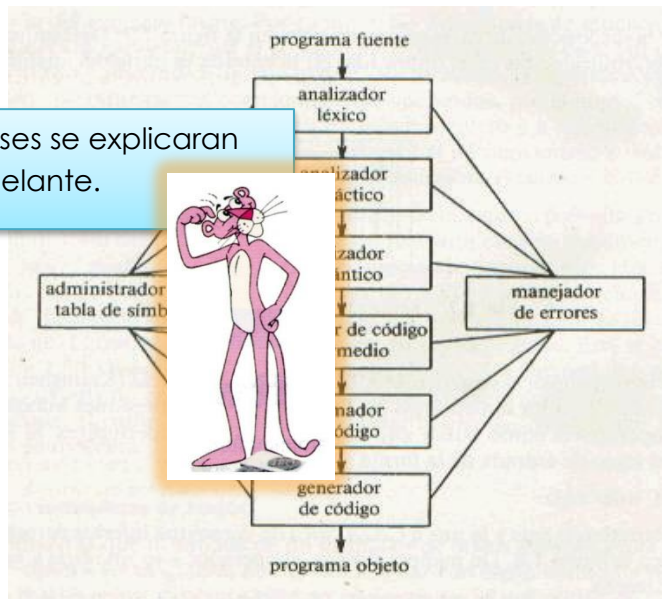


Fases de un compilador

Conceptualmente, un compilador opera en fases, cada una de las cuales transforma al programa fuente de una representación en otra. En la siguiente figura se muestra una descomposición típica de un compilador.

⁴ Book, D. (1977). Dragon Book. En A. V. Aho, *Dragon Book* (pág. 803). Dax.

Estas fases se explicaran más adelante.



Análisis Léxico:

El analizador léxico es la primera fase de un compilador. Su principal función consiste en leer los caracteres de entrada y elaborar como salida una secuencia de componentes léxicos que utiliza el

analizador sintáctico para hacer el análisis, también puede realizar ciertas funciones secundarias como eliminar del programa fuente comentarios y espacios en blanco, caracteres TAB y de línea nueva. Otra función es relacionar los mensajes de error del compilador con el programa fuente.

Esta fase se encarga de verificar si todas las cadenas pertenecen o no al lenguaje. Es decir realiza un análisis símbolo por símbolo indicando el token por cada uno de los elementos reconocidos o el error en caso de no reconocer.

Tokens y Lexemas

- ➡ Token:
 - 📄 Elemento básico del lenguaje
 - 📄 Unidad léxica indivisible
- ➡ Lexema:
 - 📄 La cadena original que se identifica como token.



Tokens y Lexemas: Ejemplos

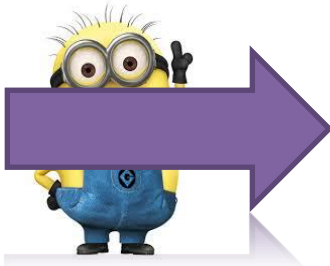
Token	Token #	Lexema
If	10	'if'
parentesis	12	'('
Identificador	1	'valor'
opSuma	30	'+'
puntoComa	48	','
Menor	56	'<'
menorigual	57	'<='

Análisis

Sintáctico:

Determina la sintaxis o estructura de un programa, la sintaxis de un lenguaje de programación se lo determina mediante reglas gramaticales de una gramática libre de

contexto similar a las ER en el análisis léxico con la diferencia que son recursivos. En esta fase se analiza la estructura de las expresiones en base a gramáticas.



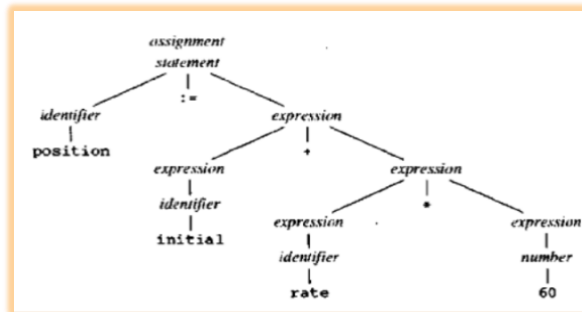
Análisis semántico:

Se trata de determinar el tipo de los resultados intermedios, comprobar que los argumentos que tiene un operador pertenecen al conjunto de los operadores posibles y si son compatibles entre sí, es decir comprobará que el significado de lo que se va leyendo es válido.

La fase de análisis semántico revisa el programa fuente para tratar de encontrar errores semánticos y reúne la información sobre los tipos para la fase posterior de generación de código.

La salida "teórica" de la fase de análisis semántico sería un árbol semántico.

Ejemplo:



Expresiones regulares:

Las expresiones regulares se usan para analizar el contenido de cadenas de caracteres por medio de patrones, a menudo llamada también **regex**, es una secuencia de caracteres que forma un patrón de búsqueda, principalmente utilizada para la búsqueda de patrones de cadenas de caracteres u operaciones de sustituciones.

Generalmente estos conjuntos se representan como:

 	Es una expresión regular que denota el conjunto vacío (el conjunto no contiene secuencias de símbolos).
	Es una expresión regular que denota al conjunto que contiene la secuencia vacía.
	Una secuencia de símbolos S es una expresión regular que denota a un conjunto que contiene a S.

Se pueden representar formalmente o mediante BNF.

BNF⁵ (slideshare, 2011)

La notación Backus-Naur (más comúnmente conocida como BNF o *Backus-Naur Form*) es una forma matemática de describir un lenguaje, esta notación nos permite definir formalmente la gramática de un lenguaje evitando la ambigüedad, definiendo claramente qué está permitido y qué no lo está.

Es un metalenguaje muy utilizado para definir la estructura sintáctica de lenguajes de programación (lenguajes formales). La regla BNF es la que sigue:

Símbolo $\rightarrow$ alternativa1 | alternativa2...

`<simbolo> ::= <expresión con símbolos>`

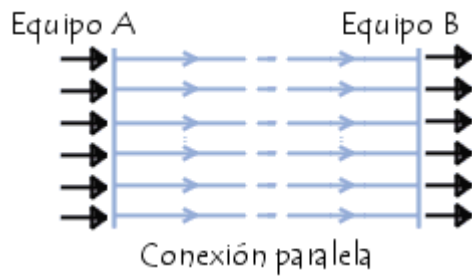


Puerto

Donde <símbolo> es un *no terminal*, y la expresión consiste en secuencias de símbolos o secuencias separadas por la barra vertical, '|', nos sirven para encerrar nombres de categorías.

paralelo:

⁵ (s.f.). Obtenido de <http://www.slideshare.net/grrodriguez/automatas-y-compiladores-analisis-sintactico>

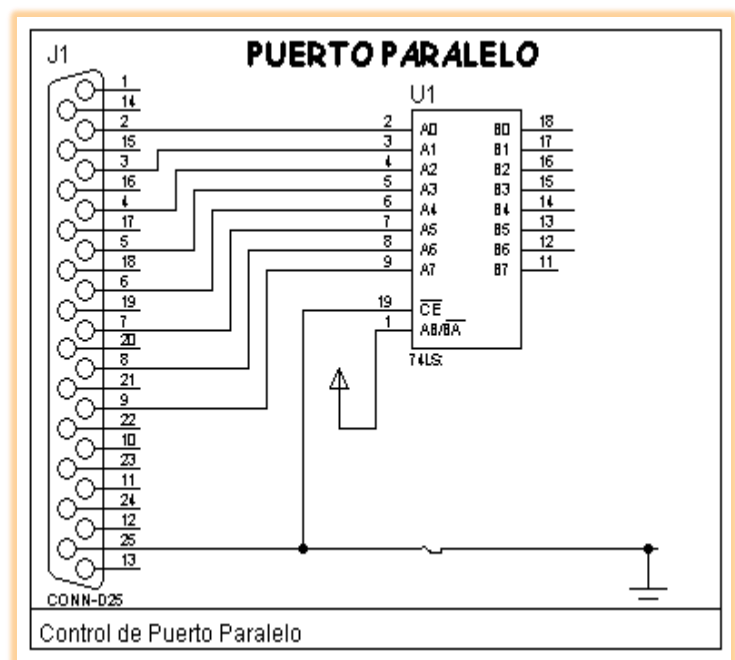


Los puertos de salida/entrada son elementos materiales del equipo, que permiten que el sistema se comuniquen con los elementos exteriores.

La transmisión de datos paralela consiste en enviar datos en forma simultánea por varios canales (hilos).



En la siguiente imagen vemos el control del PIC por medio del puerto paralelo a utilizar.



CASO PRÁCTICO

Se desea realizar un compilador para un lenguaje formal, en el cual, dicho lenguaje contenga instrucciones básicas para controlar un carrito electrónico, es decir, que desde la computadora, el usuario en un archivo de texto escriba las instrucciones que desee que haga el carro, el compilador lea el archivo, lo procese y determine si está escrito correctamente o no, y si así lo es, entonces genere salidas al puerto paralelo por cada instrucción procesada, y así el carrito las realice de manera física.

Las instrucciones que se requieren manejar son: ADELANTE, ATRÁS, LUCES y CHICHARRA.

Nota: Cabe mencionar que para las instrucciones de ADELANTE, ATRÁS, se necesita que reciban el parámetro del tiempo en segundos (enteros) en que se estará ejecutando esa instrucción, mientras que en las instrucciones de LUCES, se necesitan los parámetros de qué tipo de luz quiere ("alta o baja") y su estado ("prender" o "apagar").

En conclusión, se necesita desarrollar la ER que permita aceptar dichas instrucciones implementada en el LENGUAJE FORMAL, se genere su compilador que procese esas instrucciones para que muestre al usuario en pantalla los errores si es que los tuvo al escribir, o le muestre físicamente que las instrucciones que escribió en la computadora son ejecutadas por el carrito en el orden que las escribió con sus respectivos parámetros correspondientes.

Proyecto: Carrito-Compilador

OBJETIVO GENERAL.

El objetivo general del proyecto es realizar un compilador que analice un conjunto de instrucciones que pertenecen a un lenguaje formal, escritas por el usuario en la computadora, con las cuales se pueda programar los movimientos y acciones a realizar de un carrito electrónico conectado en la misma computadora para demostrar el funcionamiento de un compilador y una de las aplicaciones prácticas que se le puede dar.

OBJETIVOS ESPECÍFICOS

Aplicar los conocimientos de Teoría de la computación para el desarrollo de la gramática aceptada de nuestro pequeño lenguaje (Aplicadas en nuestras instrucciones).

Diseñar un lenguaje FORMAL con una gramática dependiente del contexto, para el desarrollo de nuestro lenguaje de instrucciones.

Diseñar un AFND para determinar si las cadenas de instrucciones son válidas o no de acuerdo a la gramática o ER definida por nuestro lenguaje.

Aplicar el concepto de análisis sintáctico, semántico y sintáctico (llevándolo a la programación) en el análisis del archivo de entrada para verificar si las cadenas son aceptadas o no, de acuerdo a nuestro lenguaje.

Mostrar físicamente la salida generada por nuestro compilador mostrándola a través de un carrito electrónico que obedece las instrucciones que el usuario escribió en la computadora.

Evidenciar la gran utilidad que tienen los compiladores y una de las tantas aplicaciones que se les puede dar en la vida diaria.

JUSTIFICACIÓN

Dado a la gran relación que tiene el hardware con el software, consideramos que este proyecto además de aplicar varias funciones de un compilador, permitirá mostrar un pequeño panorama de las grandes cosas que se pueden lograr con las herramientas de tecnología con las que contamos hoy en día, como lo son, las computadoras, microcontroladores, software de programación, "compiladores", y muchas otras más.

Otra motivo, es que para lograr mandarle una serie de instrucciones escritas desde la computadora al carrito, debemos realizar un análisis léxico para verificar que las instrucciones que se desean llevar a cabo pertenecen a nuestro lenguaje y respetan una gramática igualmente definida por nosotros, por lo cual, nos auxiliaremos de las etapas fundamentales llevadas a cabo por un compilador, que como sabemos, ese es el motivo de este proyecto, aplicar los conocimientos sobre compiladores a una aplicación factible y utilizable.

ALCANCES Y LIMITACIONES

El proyecto únicamente tiene planeado diseñar un pequeño lenguaje de instrucciones para manipular un carro electrónico desde la computadora, desarrollar su respectivo compilador en lenguaje C, evidenciar las principales etapas del compilador y mostrar su salida físicamente en el dispositivo electrónico, que para nuestro caso, es en un carrito con su microcontrolador.

En las limitaciones encontramos que nuestro lenguaje **tendrá muy pocas instrucciones**, para ser exactos tendrá 4 y se piensa en el transcurso agregar 2 más, por otro lado, otra de las limitaciones es que **funcionará con una comunicación alámbrica por puerto paralelo** el cual ya no es muy utilizado, sin embargo, en un futuro, se piensa adecuar el proyecto a una comunicación USB o bien, empleando comunicación inalámbrica.

DESCRIPCIÓN GENERAL DEL PROYECTO

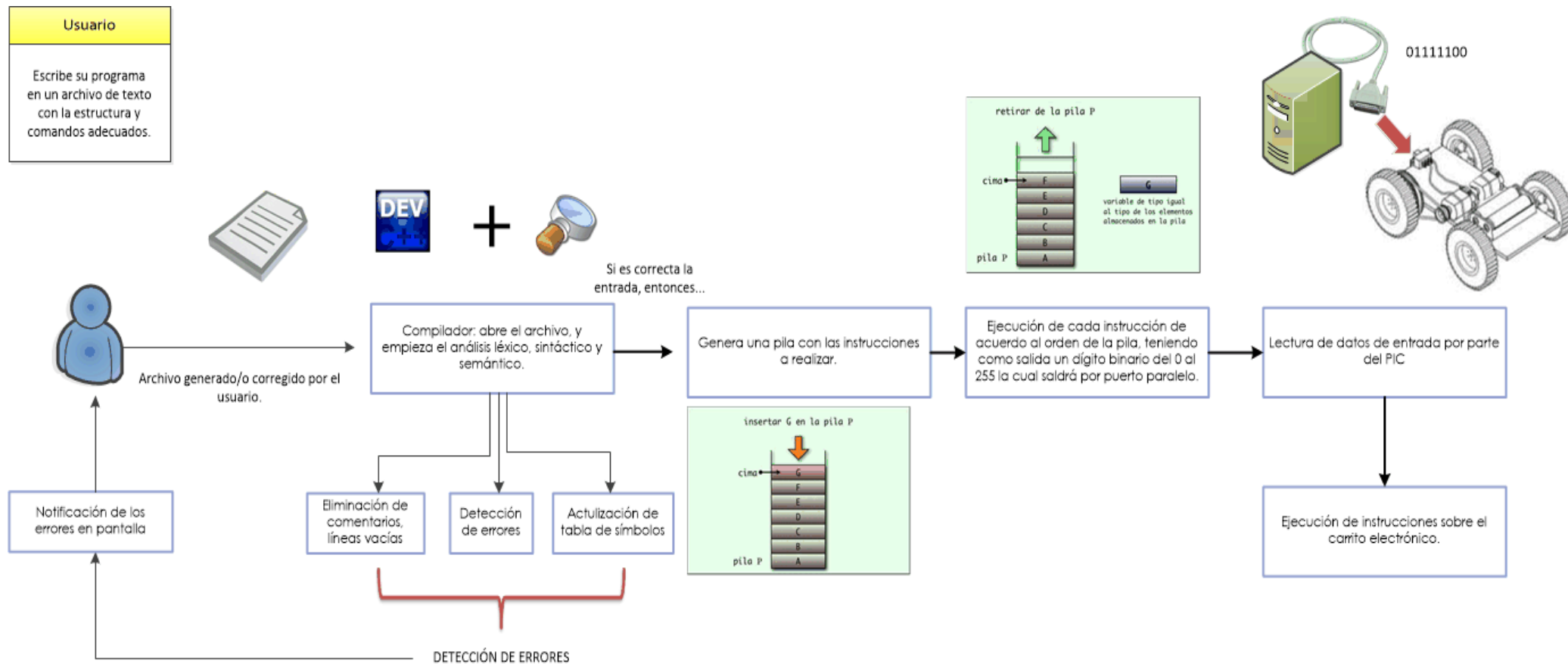
El proyecto que se desea realizar consistirá en:

- Elaborar un lenguaje sencillo que acepte cadenas que representan las instrucciones para manipular el carrito, las cuales son; AHEAD, BACK, LIGHTS y BUZZER, de tal manera que estas instrucciones podrán contener o no, ciertos parámetros.
- Desarrollar el compilador para nuestro LENGUAJE DE TIPO FORMAL que analice el archivo que el usuario haya escrito con las instrucciones que desea ejecutar, verifique si es correcto y genere la salida correspondiente.
- Que la salida del compilador sea de dos tipos, de manera visual mostrando los errores de sintaxis en pantalla, o de manera física, mandando las señales correspondientes por puerto paralelo para que éstas a su vez se dirijan al microcontrolador que tendrá el carrito y sean ejecutadas automáticamente.

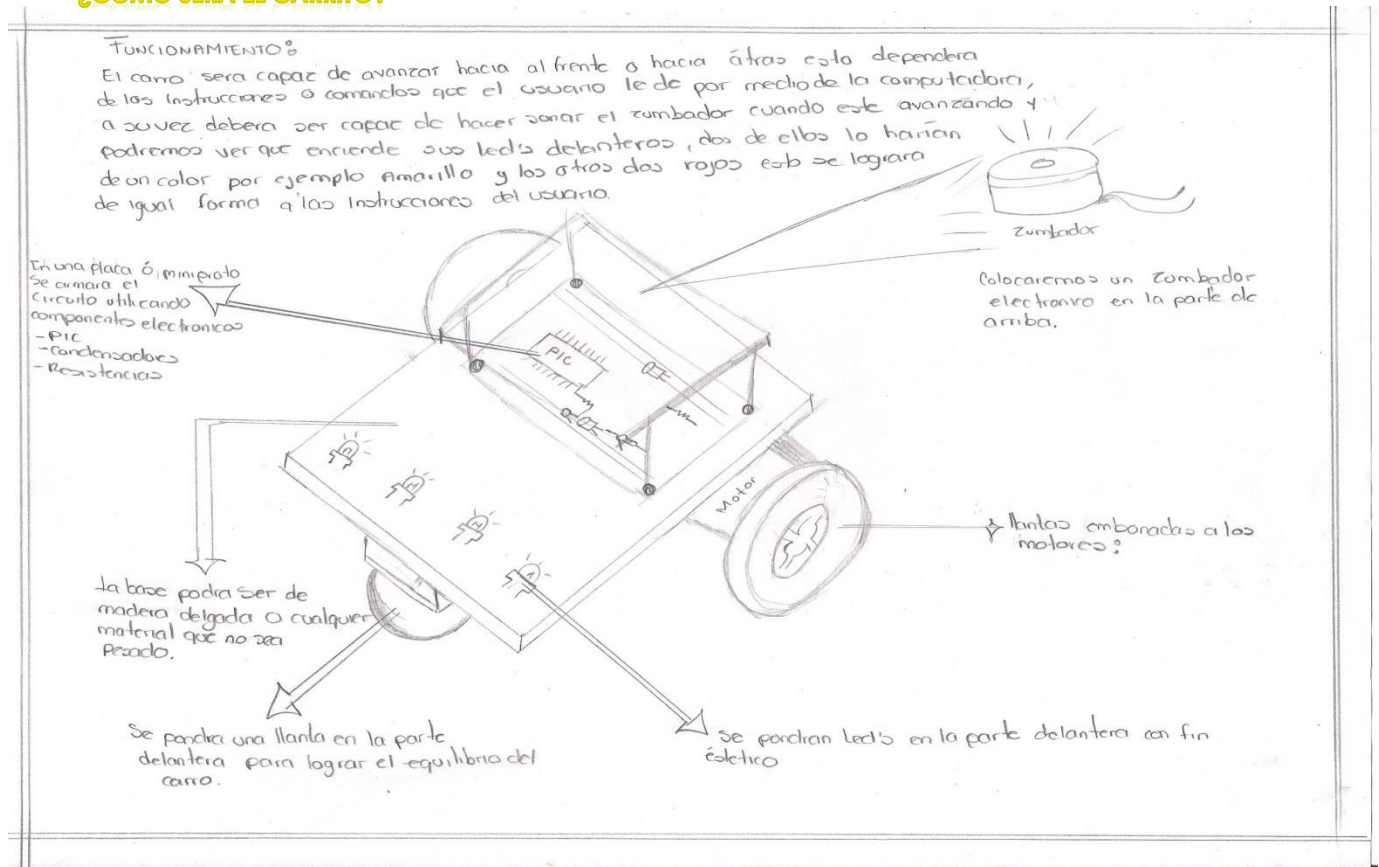
Una vez mencionado lo que se pretende hacer en este proyecto, otra parte importante es la definición de los comandos y/o instrucciones (cadenas) que serán aceptadas por el lenguaje, las cuales describimos a continuación.

Comandos aceptados	Ejemplo	Descripción
AHEAD();	AHEAD();	Este comando indicará avance en el carrito, cuando no tenga parámetros significa que sólo avanzará un segundo y cuando sí tenga parámetro, indicará el tiempo en segundos que estará avanzando.
AHEAD(Duración en segundos);	AHEAD(5);	
BACK();	BACK();	Este comando activará los motores para que el carrito retroceda, cuando no presente parámetros significa que retrocederá solo un segundo, de lo contrario, estará retrocediendo tantos segundos como se le halla asignado.
BACK(Duración en segundos);	BACK(10);	
LIGHTS ();	LIGHTS (on/off);	El comando "LIGHTS", se encargará de las luces, y puede ocurrir que:
LIGHTS ("high/low",on/off);	LIGHTS ("high",on/off);	Si recibe únicamente el parámetro de "on/off", éste actuará sobre los dos tipos de luz "high" y "low", ya sea prendiéndolas o apagándolas.
	LIGHTS ("low",on/off);	O bien, podrá recibir el tipo de luz a manipular("high/low") y el estado que se desea asignar("on/off").
BUZZER(on/off);	BUZZER(on);	Este comando permitirá encender o apagar el zumbador que traerá integrado el carrito electrónico.
	BUZZER(off);	

¿CÓMO FUNCIONARÁ?

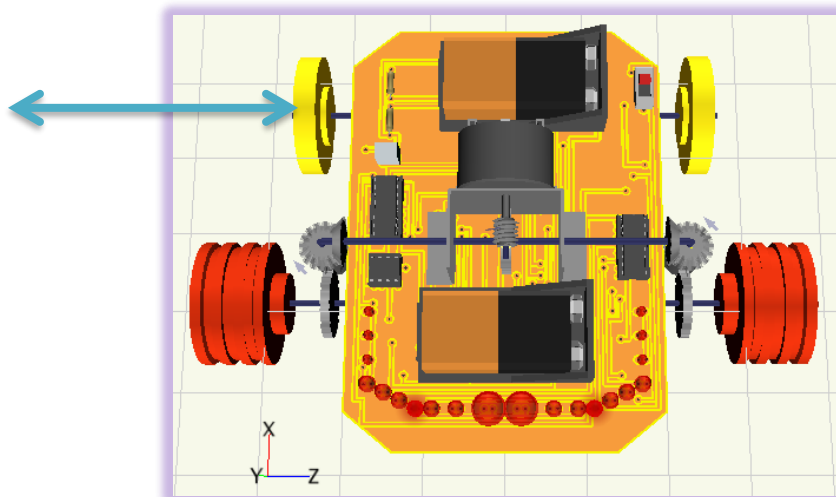


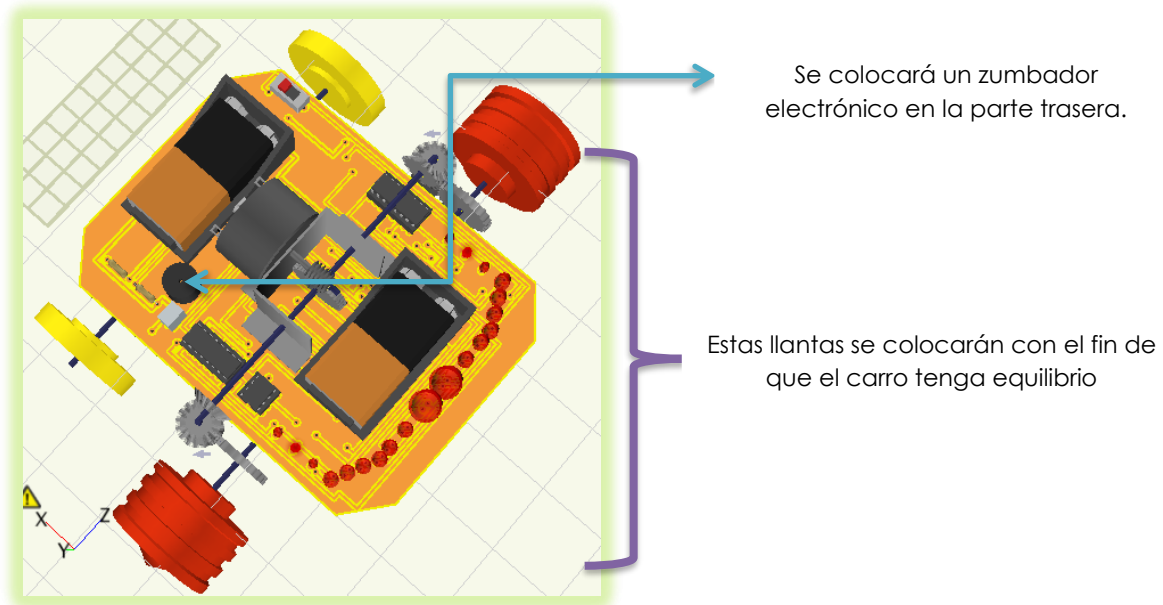
¿CÓMO SERÁ EL CARRITO?



Diseño del carrito a computadora:

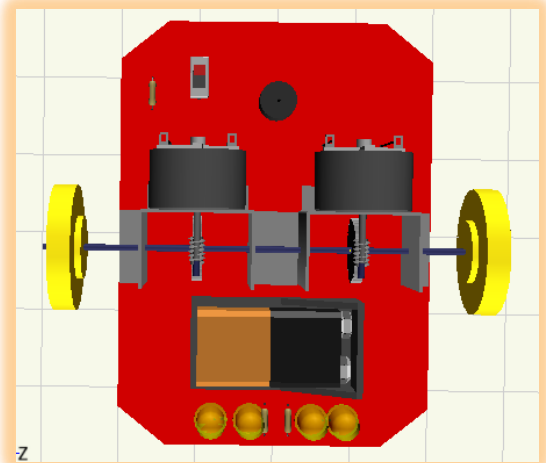
Utilizaremos componentes electrónicos como PIC, resistencias, LED'S, etc.





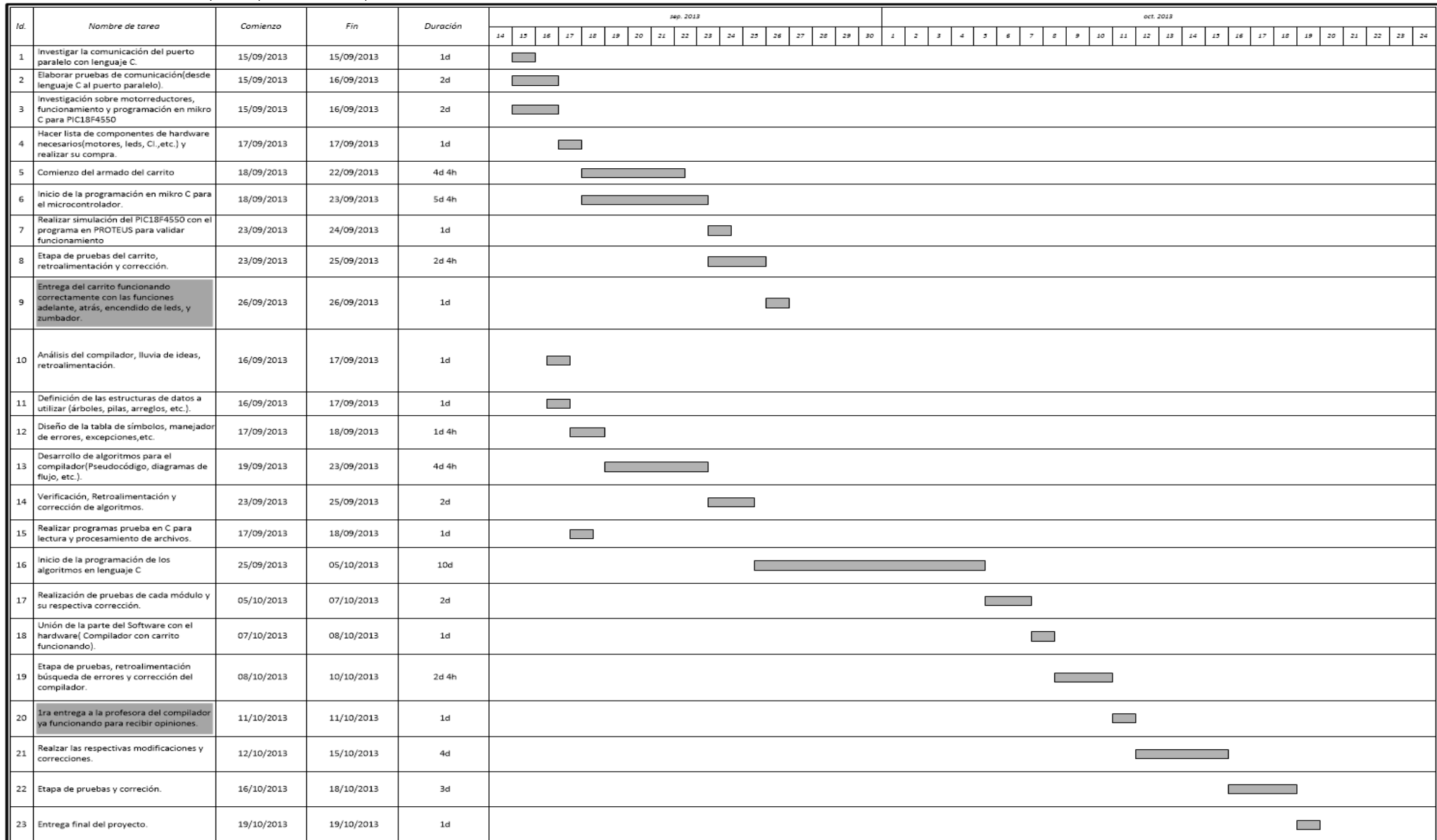
Funcionamiento:

El carro será capaz de avanzar hacia el frente o hacia atrás, esto lo haremos gracias a las instrucciones que el usuario le introducirá por medio de la computadora, las cuales serán analizadas y procesadas para que el carro realice lo que el usuario le pidió, también podrá hacer sonar el zumbador electrónico cuando esté avanzando.



¿CÓMO SE LLEVARÁ A CABO?

Dado a las diversas tareas que se han de realizar, se ha planteado una gráfica de Gantt para tener un control de las actividades, el tiempo que se tiene para cada una de ellas y el responsable o responsables de cada tarea.



El reparto de actividades será de la siguiente manera.

No. de tarea	Responsable	Integrantes:
1	Hugo	Castañeda Ibáñez Hugo Erik
2	Leslie	Gutiérrez Gutiérrez Leslie
3	Montserrat	Mendieta Rojas Delfino
4	Montserrat y Leslie	Silva Maya Angelli Monserrat
5	Monserrat y Hugo	
6	Monserrat y Hugo	
7	Delfino	
8	Todos	
9	Todos	
10	Todos	
11	Todos	
12	Delfino y Leslie	
13	Todos	
14	Todos	
15	Hugo	
16	Todos	
17	Monserrat y Leslie	
18	Delfino	
19	Todos	
20	Todos	
21	Delfino y Leslie	
22	Todos	
23	Todos	

En la tarea Número 4, la cual es la compra de los componentes a utilizar se estima un gasto de \$250, esto debido a los siguientes materiales que hacen falta.

Componentes	Costo aproximado
2 motorreductores de 6 V	\$100
2 llantas de carrito	\$60
1 C.I. L293D	\$10
2 Pilas cuadradas 5V	\$40
1 Base de carrito	\$30
Total	\$240

NOTA: Algunos otros componentes necesarios no será necesario su compra ni gasto adicional dado a que se cuenta con ellos desde el semestre anterior que se llevó la materia de dispositivos programables.

¿QUÉ HERRAMIENTAS UTILIZAREMOS?

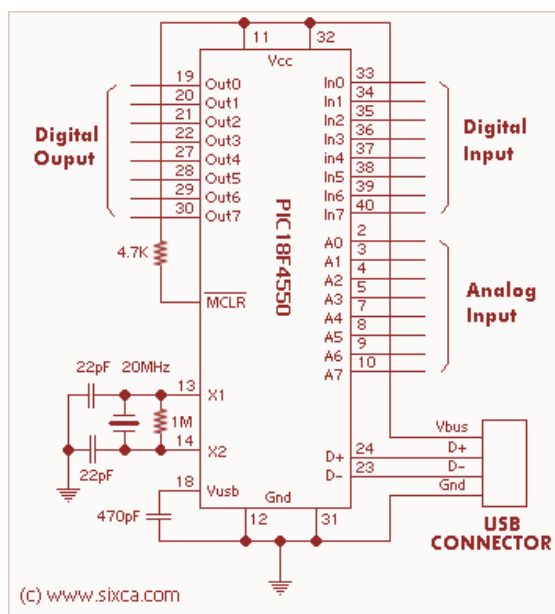
- Para el desarrollo del compilador utilizaremos **DEV C++**
- Para la programación del PIC18F4550 utilizaremos **"mikro C PRO for PIC"**
- **Simulador Proteus.**
- **PICKit2**, necesario para grabar al PIC con el programa generado.
- **Microcontrolador PIC18F4550**

PIC 18F4550 JUSTIFICACIÓN DE USO

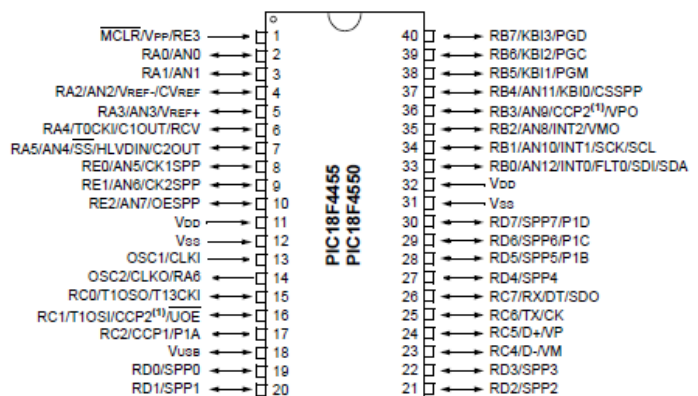
Para poder llevar a cabo nuestro proyecto, necesitamos como herramienta el PIC, ya que en cursos anteriores (Dispositivos Programables y Sistemas Digitales) hemos adquirido los conocimientos suficientes para llevar a cabo esta labor, teniendo en cuenta todo esto vamos a describir de manera general el dispositivo a utilizar.

CARACTERÍSTICAS

PUERTOS DEL PIC 18F4550



40-Pin PDIP



La imagen muestra las entradas y salidas análogas y digitales del dispositivo, como se observa cuenta con 40 pines los cuales se dividen en puertos que van del A al E, así como los puertos que sirven para alimentar de corriente al PIC.

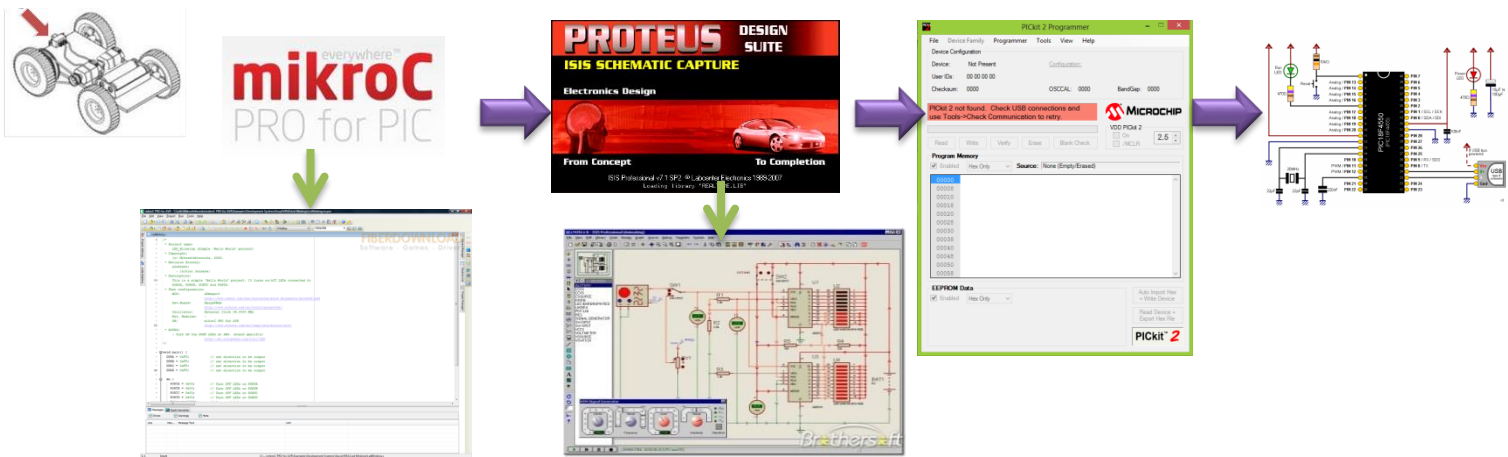
También cada puerto tiene un funcionamiento diferente dependiendo del número de pin al que se esté usando; nosotros vamos a hacer uso del puerto A o B según nos convenga tomando en cuenta que este dispositivo también cuenta con un puerto paralelo para la transmisión de datos como se muestra en la siguiente tabla.

Esta es una tabla con algunas de las características más importantes:

Características	Pic18F4550
Frecuencia de operación	Hasta 48MHz
Memoria de programa(bytes)	32.768
Memoria RAM de datos (bytes)	2.048
Memoria EEPROM datos	256
Interrupciones	20
Líneas de entrada/ salida	35
Temporizadores	4

INTERACCIÓN ENTRE LAS HERRAMIENTAS

La manera en que se utilizarán las herramientas será de la siguiente manera:



Cuando ya se tenga el carrito armado, se procederá al desarrollo del programa que tendrá el PIC para realizar las acciones que queremos, el programa se hará utilizando mikroC PRO for PIC.

La evidencia que generará será un código fuente que será grabado en el PIC.

ENTREGABLE: Un código fuente en mikro c.

En PROTEUS lo que se hará es armar el circuito que tendrá el carrito y se probará el código previamente generado en mikroC para verificar que realmente el programa está bien y ver el comportamiento que tienen los componentes y así estar seguros de que al armarlo estará funcionando de manera correcta.

En concreto, en PROTEUS se hace la simulación de cómo funcionará el circuito con el código fuente programado.

ENTREGABLE: Una simulación en PROTEUS de lo que harán los componentes del carrito.

Una vez que se tiene el código fuente y se verificó su correcto funcionamiento en PROTEUS, se utilizará el programa PICKit 2 para grabar el programa en el PIC18F4550 de manera real, para así estar listo a recibir entradas del puerto paralelo y realizar las acciones correspondientes.

ENTREGABLE: El PIC18F4550 con programa grabado y funcionando conjuntamente con su circuito, listo para recibir las salidas del puerto paralelo, procesarlas y ejecutarlas.



En Dev C++ simplemente se hará la programación del compilador, desde el cual, se gobernará las salidas del puerto paralelo.

ENTREGABLE: El programa compilador que podrá analizar el código fuente escrito por los usuarios y mandará las salidas al puerto paralelo.

¿CÓMO SERÁ LA ENTREGA FINAL?

Para la entrega final, como resultado del proyecto se tendrá que tener los siguientes entregables:

- Programa compilador hecho en lenguaje C para nuestro lenguaje diseñado.
- Carrito electrónico funcionando correctamente, con las acciones de avanzar, retroceder, prender y apagar su chicharra y sus luces, lo cual obviamente implica su respectivo programa grabado en el PIC.

La manera de evidenciar el funcionamiento será de la manera siguiente:



Para:

- AHEAD(10); acción: El carro avanzará 10 segundos
- BACK(5); acción: El carro retrocederá por 5 segundos
- LIGHTS("low",on); acción: Las luces bajas se prenderán
- BUZZER(on); acción: Encenderá su chicharra
- AHEAD(5); acción: Avanzará por 5 segundos
- LIGHTS("low",off); acción: Apagará sus luces bajas
- BUZZER(off); acción: Apagará su chicharra

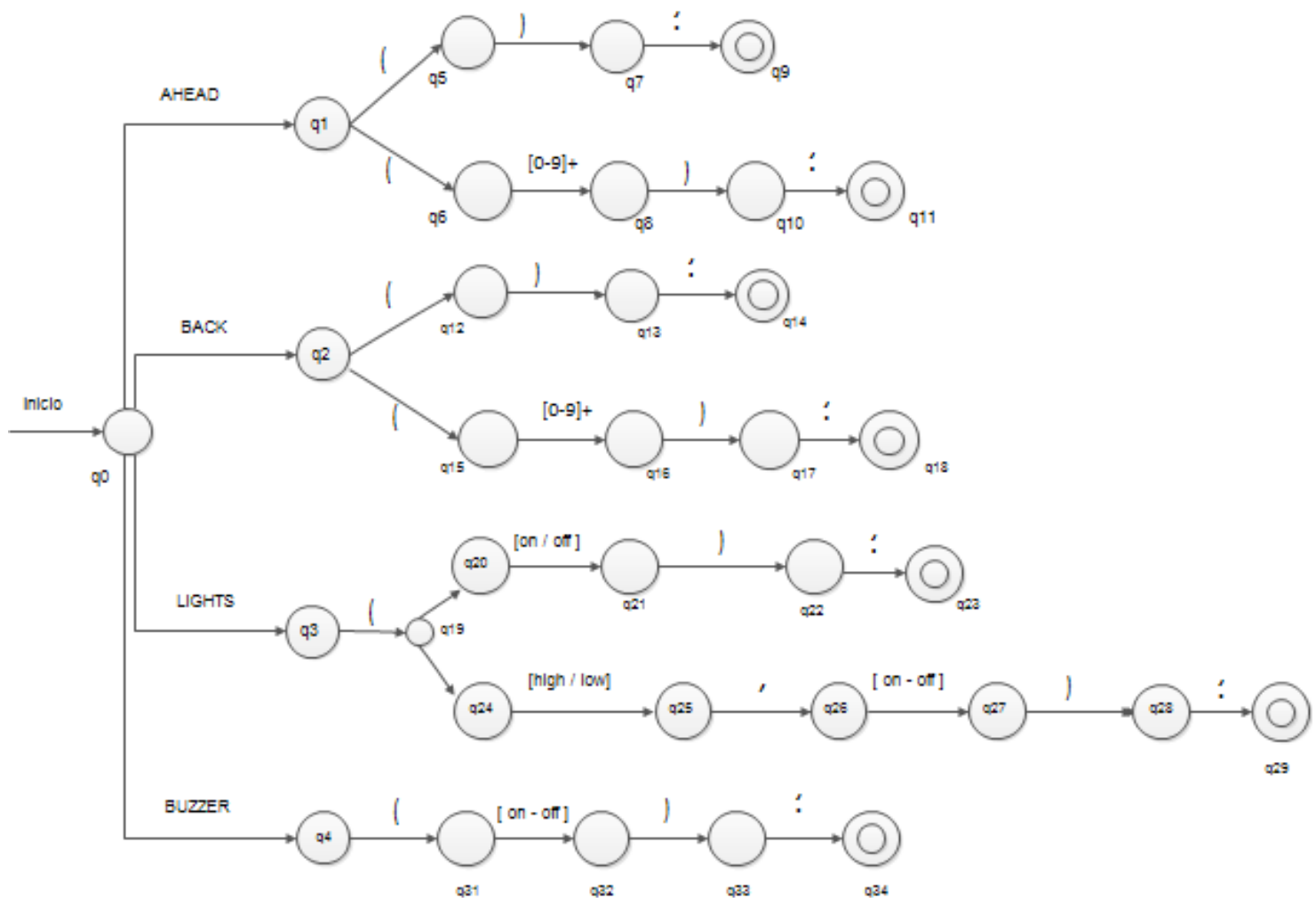
}

NOTA: Cada instrucción se realiza una por una, es decir, hasta que acabe de avanzar 10 segundos, hará la siguiente instrucción y así sucesivamente.

De esta manera se evidenciará que el carrito realmente hace lo que esté escrito en el lote de instrucciones que el usuario escribió, y obedece a la salida del compilador.

DISEÑO DEL AFND PARA LAS INSTRUCCIONES

Dado que los comandos serán escritos por el usuario, éstas deberán ser analizadas utilizando expresiones regulares, las cuales serán obtenidas a partir de este autómata que permite validar si es una instrucción del lenguaje y si el formato que tienen es el correcto para poder ser ejecutada además de determinar si hay errores y en donde están, etc.



BNF DE COMANDOS ACEPTADOS**(Backus-Naur Form)****AHEAD();**

<AHEAD>::= AHEAD/ahead <Sin parámetros de tiempo>

<Sin Parámetros de tiempo>::= < $\emptyset$ >

AHEAD (Duración en segundos);

<AHEAD>::= AHEAD/ahead <Parámetros de tiempo> / <Segundos>

<Parámetros de tiempo>::= <Segundos> < $\emptyset$, 1, 2, 3, ..., n >

<Segundos>::= < $\emptyset$, 1, 2, 3, ..., n >

BACK();

<BACK>::= BACK/back <Sin parámetros de tiempo>

<Sin parámetros de tiempo>::= < $\emptyset$ >

BACK(Duración en segundos);

<BACK>::= BACK/back <Parámetros de tiempo> / <Segundos>

<Parámetros de tiempo>::= <Segundos>

<Segundos>::= < $\emptyset$, 1, 2, 3, ..., n >

LIGHTS();

<LIGHTS>::= LIGHTS/lights <Parámetros> | <on/off>

<Parámetros>::= <on/off>

<on>::= <1>

<off>::= <0>

LIGHTS("high/low",on/off);

<LIGHTS>::= LIGHTS/lights <Parámetros> | <"high/low"> <on/off>

<Parámetros>::= <"high"> <on>/<off>

<"high">::= <on>/<off>

<Parámetros>::= <"low"> <on>/<off>

<"low">::= <on>/<off>

BUZZER (on/off);

<BUZZER>::= BUZZER/buzzer <Parámetros> | <on>/<off>

<Parámetros>::= <on>/<off>

<on>::= <1>

<off>::= <0>



Manual del usuario

El desarrollo del proyecto cuenta con una serie de funciones sencillas con las cuales alguno de los padres o ambos podrán programar a este pequeño auto para ayudarlos a cumplir la labor de cuidar del bienestar de su hijo o hija. Las funciones con las que cuenta este proyecto son:

- AHEAD();
- BACK();
- LIGHTS();
- BUZZER();



Las especificaciones de las funciones anteriores se describen a detalle en la siguiente tabla:

Comandos aceptados	Ejemplo	Descripción
AHEAD();	AHEAD();	Este comando indicará que el carrito tiene que avanzar, cuando no tiene parámetros significa que sólo avanza un segundo y cuando tiene parámetro, éste indicará el tiempo en segundos en que el carrito estará avanzando.
AHEAD(Duración en segundos);	AHEAD(5);	
BACK();	BACK();	Este comando activará los motores para que el carrito retroceda, cuando no presente parámetros significa que retrocederá solo un segundo, de lo contrario estará retrocediendo tantos segundos como se le halla asignado.
BACK(Duración en segundos);	BACK(10);	
LIGHTS();	LIGHTS (on/off);	El comando "LIGHTS", se encargará de las luces, y puede ocurrir que:
LIGHTS ("high/low",on/off);	LIGHTS ("high",on/off);	Si recibe únicamente el parámetro de "on/off", éste actuará sobre los dos tipos de led("high/low").
	LIGHTS ("low",on/off);	O solo puede recibir el tipo de luz a manipular("high/low") y el estado que se desea asignar("on/off").
BUZZER(on/off);	BUZZER(on);	Este comando permitirá encender o apagar el zumbador que traerá integrado el carrito electrónico.
	BUZZER(off);	

Compiladores, Principios, Técnicas y Herramientas

Alfred V. Aho y Jeffrey D. Ullman

Dragon Book

Número de páginas. 803

Título: COMPILADORES, TRADUCTORES Y COMPILADORES CON
LEX Y YAC, JFLEX/CUP Y JAVA CC

Autor: Sergio Gálvez Rojas y Miguel Ángel Mora Mata

Editorial: Universidad de Málaga

Páginas: 319

Comunicación con el puerto paralelo: <http://mimosa.pntic.mec.es/~flarrosa/puerto.pdf>